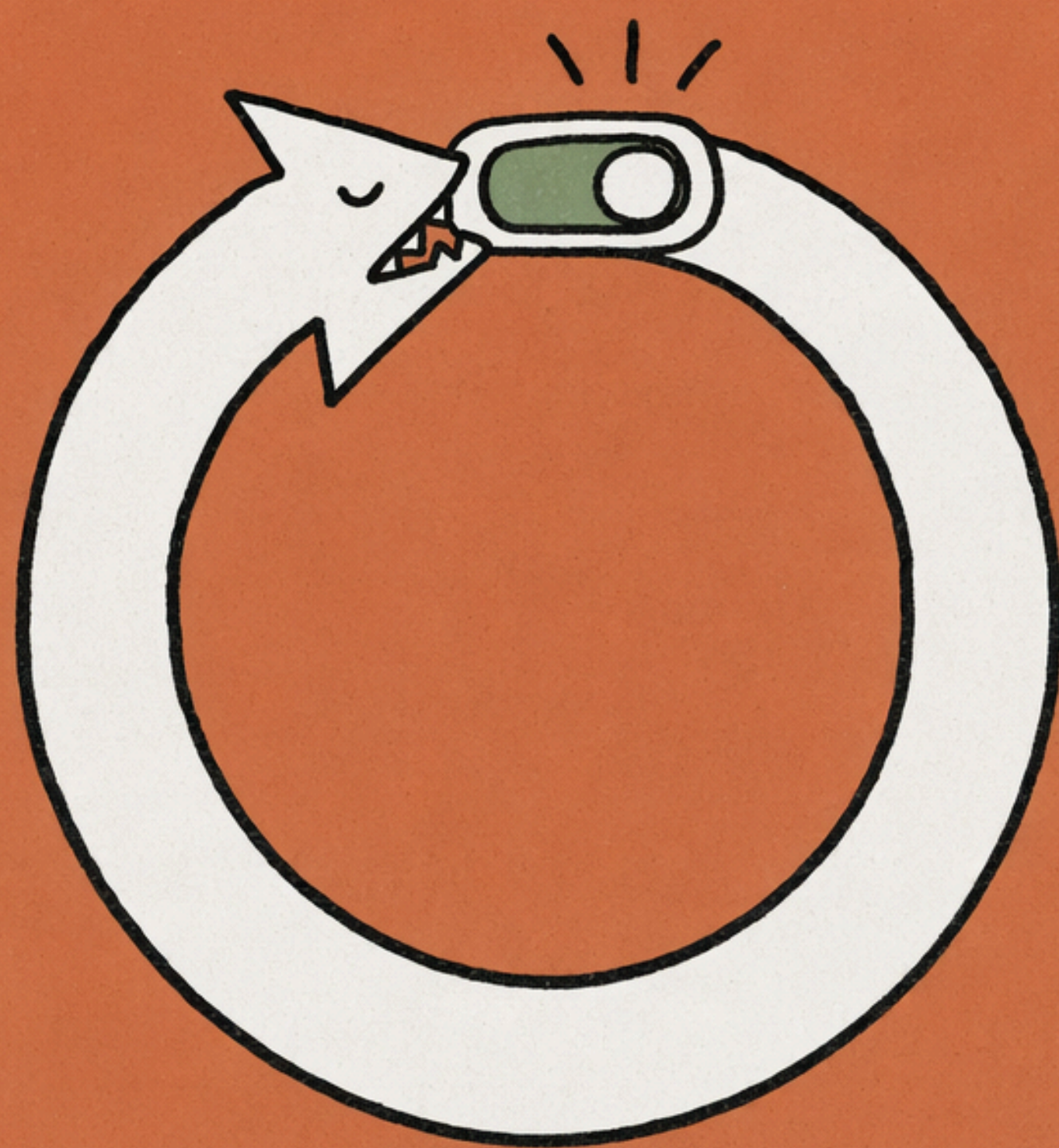




💡 Agent · Loop Engineering 8,296 字 约 17 分钟

关于 Loop Engineering 的 6 问 6 答

Loop Engineering 6 月份才提出的概念，现在才 1 个月半月，但感觉已经提出了很久很久了，真是 AI 一年，人间三年。

后知后觉，这篇 6 问 6 答说清楚它为什么突然火了、到底是什么、又该怎么用。

Q1: Loop Engineering 是怎么火起来的？

- 今年 6 月 2 日 Claude Code 之父 Boris Cherny : "I don't prompt Claude anymore. I have loops that are running."
- 2026 年 6 月 7 日，Peter Steinberger (OpenClaw 的作者，2026 年 2 月加入 OpenAI) 发了条他自称"每月一次的提醒": "you shouldn't be prompting coding agents"

anymore. You should be designing loops that prompt your agents."

- 同一天，Google Chrome 的 Addy Osmani 把这个想法写成了一篇文章《Loop Engineering》。

事实上，追述到 2025 年 7 月，Geoffrey Huntley 就开始推广 **Ralph Wiggum Loop**。最原始的形式甚至就是把 Claude Code 放进一个 shell 循环里，不断让它继续做任务、读取反馈、修改代码：**目标 → 执行 → 看结果 → 修正 → 再执行**。

他后来让 Claude 在这种循环里连续工作数月，做出了一个完整的编程语言项目。这件事让很多开发者第一次直观意识到：> 真正有价值的可能不是“写一个完美 Prompt”，而是让 Agent **持续工作直到目标达成**的机制。

但当时大家更多叫 **Ralph、agent loop、autonomous coding**，还没有统一成“Loop Engineering”这个概念。

一个词能火背后总得有真东西撑着。我觉得是两件事凑齐了：

1. 模型智能的提升突破某个临界点，真能自己干长程任务了。模型的智能水平是基础。
2. 工具把循环变成了一条命令。Claude Code、Codex 直接把 **/loop**、**/goal** 做成了内置命令，门槛一下就下来了。

一个是本质，本质是模型的智能突破了某个水平，另外一个产品端，anthropic和openai 为了推广它们的强模型，用了简单的内置命令把这些模型的场景进一步开拓，顺便也让开发者能非常直接轻易地用上loop的功能。

Q2：它跟 **Prompt**、**Context**、**Harness Engineering** 有什么关系？

《四层工程进化论》把这层关系讲得最清楚：随着模型能力越来越强，会诞生出更高的任务抽象：

- **Prompt Engineering (2023)**：模型那会只能做简单问答，你能做的就是把一次输入调到最好，让AI给个好答案。
- **Context Engineering (2025 年)**：随着模型性能越来越强，从单轮问答变成可以做简单的多轮agent任务。难点变成"怎么控制agent的上下文，最可能触发模型的期望行为"，RAG、MCP、message history 管理，都是控制模型在agent的交互中每次能看到什么，让它更好地完成agent任务。
- **Harness Engineering (2026 年初)**：要解决的是"怎么让减少错误重犯"——不再指望模型"看到正确信息就会做对"，而是搭一套 linter、test gate、observability，把agent执行任务更加稳定。

- **Loop Engineering (2026 年中)**: 这一层管的是"谁来触发下一步、谁来判断做完了", 不再是人天天分配任务, 而是设计一套自己发现活、自己推进、自己收尾的系统。

这四层为什么会一层一层往外冒, 说穿了只有一个驱动力: **模型智能变强, 能扛的任务规模变大了**, 没有别的更复杂的原因。

- 模型只够答一道题的时候, 你唯一能做的就是把这题问好——于是有了 **Prompt Engineering**。
- 模型能自己调工具、跑几步简单的agent任务, "prompt engineering"不够用了, 难点变成在一个agent的工作流中, "模型每一步该看到什么正确信息、减少干扰信息"——于是有了 **Context Engineering**。
- 模型能不用人盯着、连续跑几个小时甚至几天的时候, "喂对信息"也不够了, 得防着它在没人看的这几个小时里把一步小错滚成一堆烂摊子——于是有了 **Harness Engineering**。
- 模型强到能自己判断"这活儿现在该不该我来发起、算不算做完了", 不用人天天分配任务的时候——于是有了 **Loop Engineering**。

每一层都是被上一层的瓶颈逼出来的, 不是凭空多长出一层。

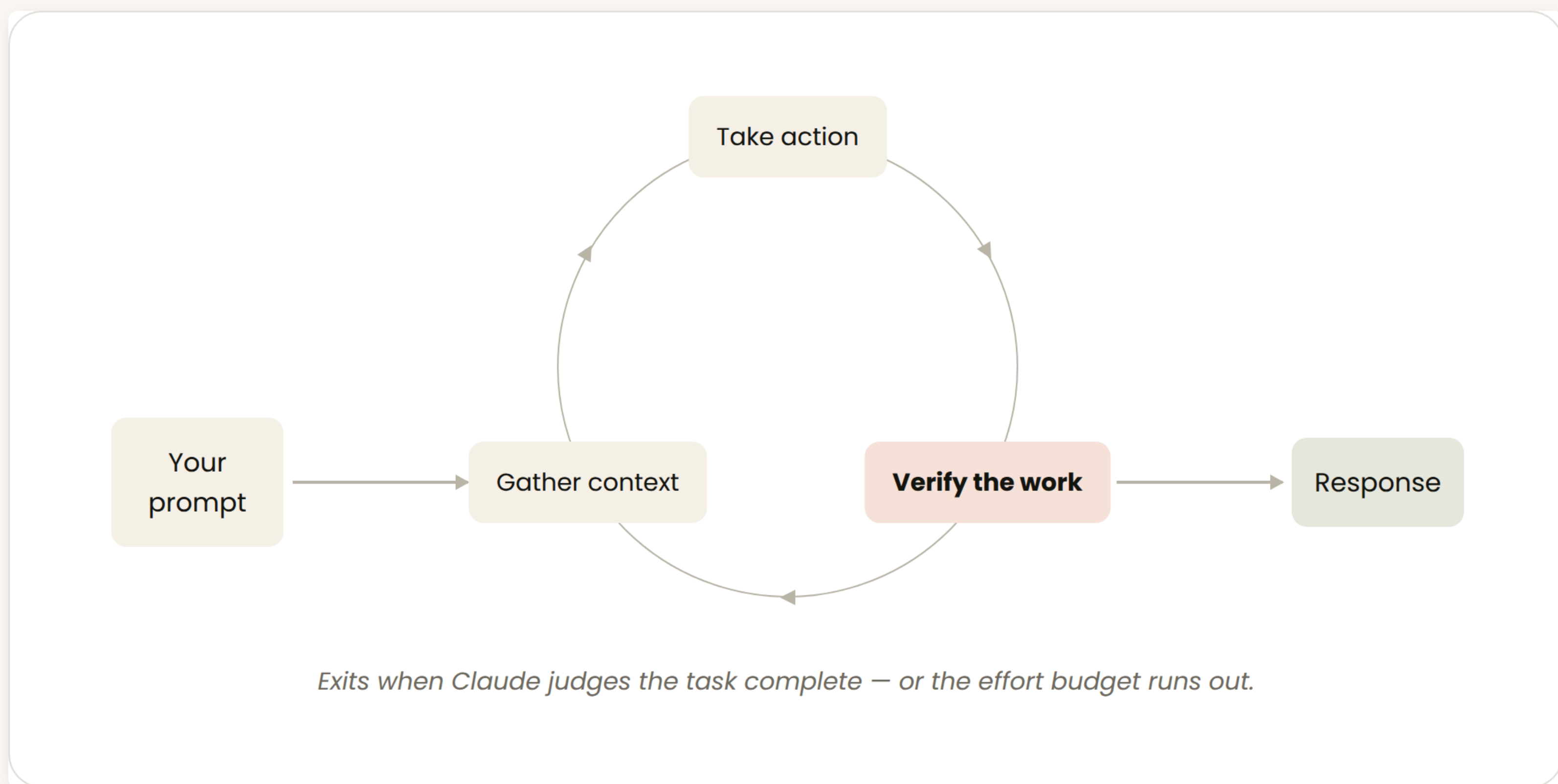
一层包一层, prompt engineering → context engineering → harness engineering → loop engineering, 反过来说, **prompt 写得模糊、context 配置混乱, Harness、Loop 搭得再漂亮也救不了**, 因为循环每一轮真正喂给模型的, 还是最底层prompt和context。

Q3: 到底什么才算一个 loop?

我一开始也没搞懂"loop"指的是啥。尤其 Claude Code 这大半年一口气上线了一堆相关功能——**/loop** (2026-03-07)、**/schedule** (2026-04-14, routines 功能的一部分)、**/goal** (2026-05-11)、**/workflows** (2026-05-28, dynamic workflows, 目前还是 research preview) ——看得人眼花。后来想明白: **这些其实全是 loop, 只是触发方式不一样**。

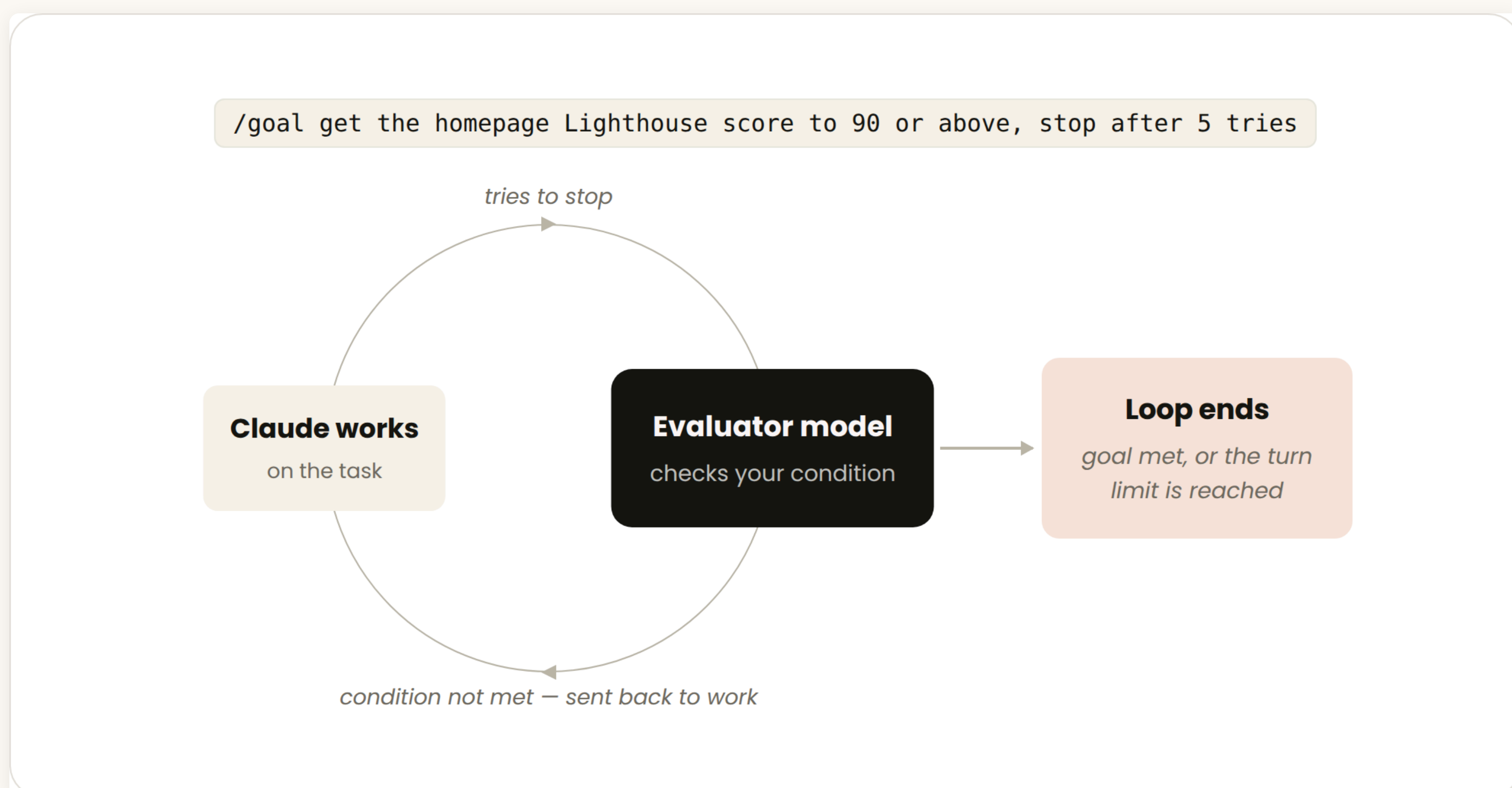
后来看了一些资料后, 想清楚了, 判断标准很简单: 只要AI在重复同一个cycle——**做一步 → 看结果 → 决定下一步 → 再来一轮**——就是个 loop。剩下的区别只在于"谁来发起下一轮"。Anthropic 《Getting started with loops》给了一套官方分类, 拿它对一下 Claude Code/Codex 的命令就清楚了:

- **单轮自己判断 → Turn-based**。最基础的一种: 一句 prompt 丢过去, AI开始干活, 干完后, 自己验证, 判断干完了就退出, 判断没干对, 那就继续干。



最基础的单轮循环：拿到 prompt → 收集上下文 → 行动 → 自查 → 出结果，Claude 自己判断"完了没"（图源：Anthropic 《Getting started with loops》）

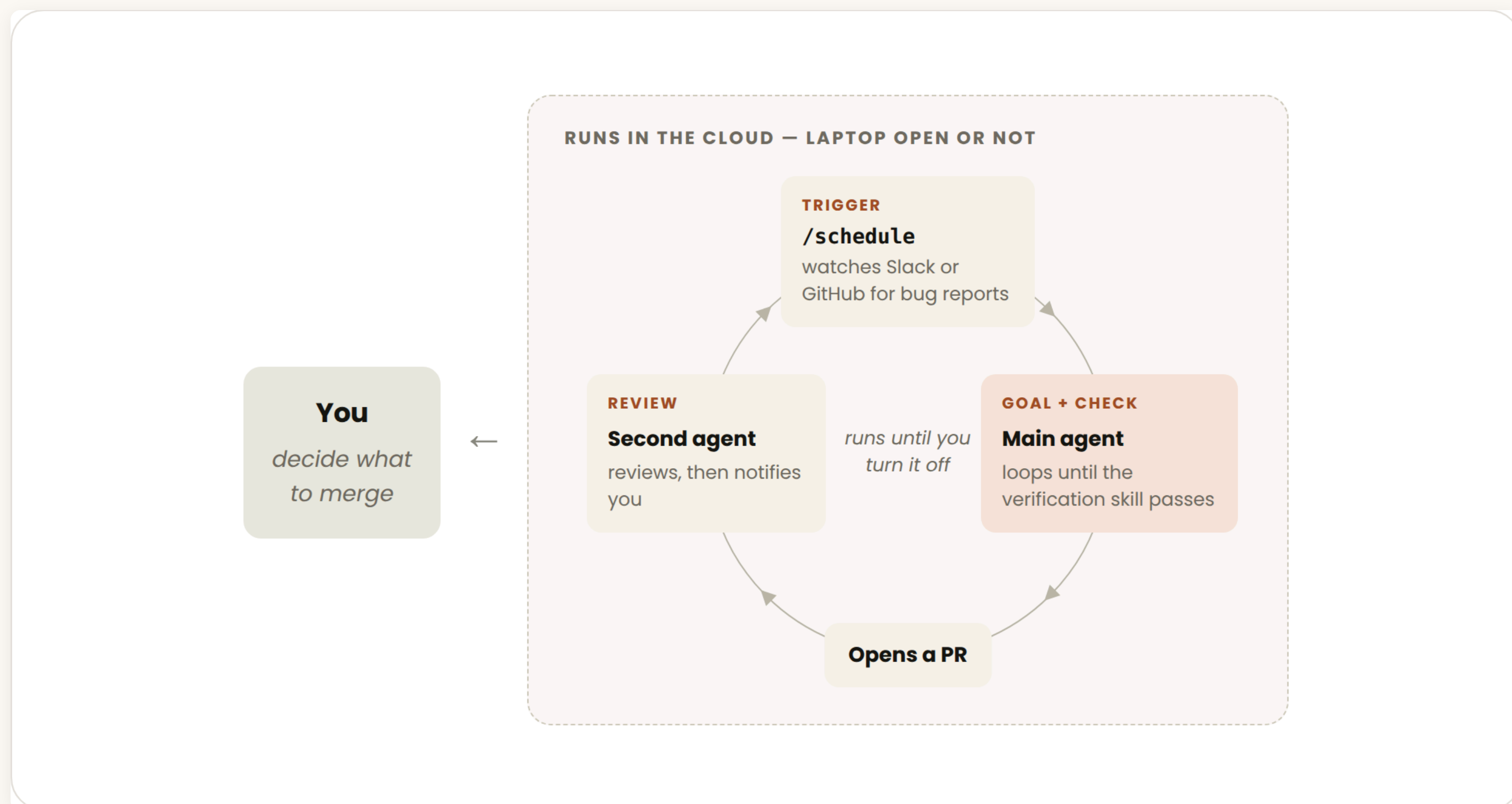
- **奔着目标去** → **/goal**（Claude Code/Codex 都有）。给个目标，模型自己规划、执行、自查、纠错，干到目标达成或者撞上轮数上限才停。这是最像 agent 的一种循环。我理解跟上面的turn-based一个大的区别是，判断的模型是另起的一个模型，拥有独立的上下文，这部分甚至可以用一个强模型，而不是自己做完自己检。



/goal 的运作方式：Claude 干活 → evaluator 模型对着你给的条件打分 → 没过就带着反馈打回去重做，直到条件满足或轮数用完（图源：Anthropic 《Getting started with loops》）

- **按周期跑** → **/loop** 和 **/schedule**（两家都有；Anthropic 官方把这两个归成同一类"time-based loop"）。每隔一段时间重复跑一个 prompt，跑完睡一会儿再跑，直到你喊停或者活干完了。它是"盯着钟看某个东西"，不是冲终点去，正好跟 **/goal** 相反——不用等事件发生，也没有"做到什么程度算完"这种目标判断。最常见的例子就是每天/每周定时总结过去一段时间的用户反馈、跑一遍数据周报，这类活既不需要人触发，也谈不上"达成目标"，跑完这一轮，等到下个周期继续跑就行。Codex 这边对应的命令叫 **/cron** 还在排期，云端定时走的是 Automations。

- **无人值守、自己找活干 → Proactive loop**。事件或时间触发，全程不用人在场，直到你把它关掉。官方给的例子是把 `/schedule`、`/goal`、skill、dynamic workflows、auto mode 拼在一起，让系统自己发现 issue、分派、修、复核。



*Proactive loop 的一个真实例子：`/schedule` 每小时检查一次 `#project-feedback` 中的 bug 报告。`/goal`：在本轮发现的所有报告都完成分类、处理并回复之前，不要停止。修复 bug 时，使用工作流在三个并行 worktree 中分别探索三种解决方案，并让一个 judge 对这些方案进行对抗性评审。

以上这些都是loop engineering。

Q4: Loop Engineering 有哪几大要素？

把几篇博客的清单凑一起，是两组东西，各有出处。

第一组，让循环能跑起来的"零件"——Requesty 和 Addy Osmani 列的清单几乎一样，Osmani 自己的说法是"五个原语 + 一个状态存储"：

- Automations（自动化心跳）：定时触发或事件触发，让 agent 自己开始干活，不用你手动敲一句 prompt。这是让"跑一次"变成"loop"的关键——少了它，前面的都只是单轮调用。
- Worktree（隔离工作区）：每个 agent 在独立的 git worktree 里跑，互不踩脚，也护住主分支。并行时尤其要紧。
- Skills（技能）：把项目知识写进 `SKILL.md`，agent 自动取用，不用每轮重新教一遍。
- Connectors / MCP（连接器）：接到 issue tracker、数据库、Slack 这些已有系统上。
- Subagents（子代理）：按角色拆开——explorer、implementer、verifier 各管一摊，各有各的上下文。写代码的和验代码的分给不同 agent。

以上五个是"原语"，State（状态记录）**是单独拎出来的"+1"：模型每轮都会忘，但仓库不会——用 markdown、Linear 看板这类外部记录当记忆，记下"做到哪了"，下一轮才不会白

干。

第二组，让循环转得对的"规矩"——Anthropic 官方博客和腾讯云那篇讲得更实在

- **成功和终止条件要写死，别留"做好为止"**：官方原话是"Be specific about what done looks like so Claude can arrive at the solution sooner"。/goal 这类目标型循环，官方给的例子直接带轮数上限——"stop after 5 tries"。没有硬上限，循环要么提前收工，要么在原地打转到你手动喊停。
- **验证不能自己批自己，靠"新鲜视角"**：官方原话是"A reviewer with fresh context is less biased"——审查的 agent 不共享写代码那个 agent 的推理过程，才不容易被带偏。/goal的验证就是单独的上下文的。
- **错一次不能只改一次，得把教训焊进系统里**：官方原话是"don't stop at fixing the individual issue, try to encode it to improve the system for all future iterations"。Agent 犯错 → 分析根因 → 写进 AGENTS.md 或加一条 CI check 或变成一个skill，这其实是harness engineering的体现。
- **循环别开太勤**：官方原话是"Don't run routines more often than you need to: match the interval to how often the thing you're watching changes"——纯粹是为了省 token，巡检频率跟着"你盯的东西变化有多快"走，不是越频繁越好。

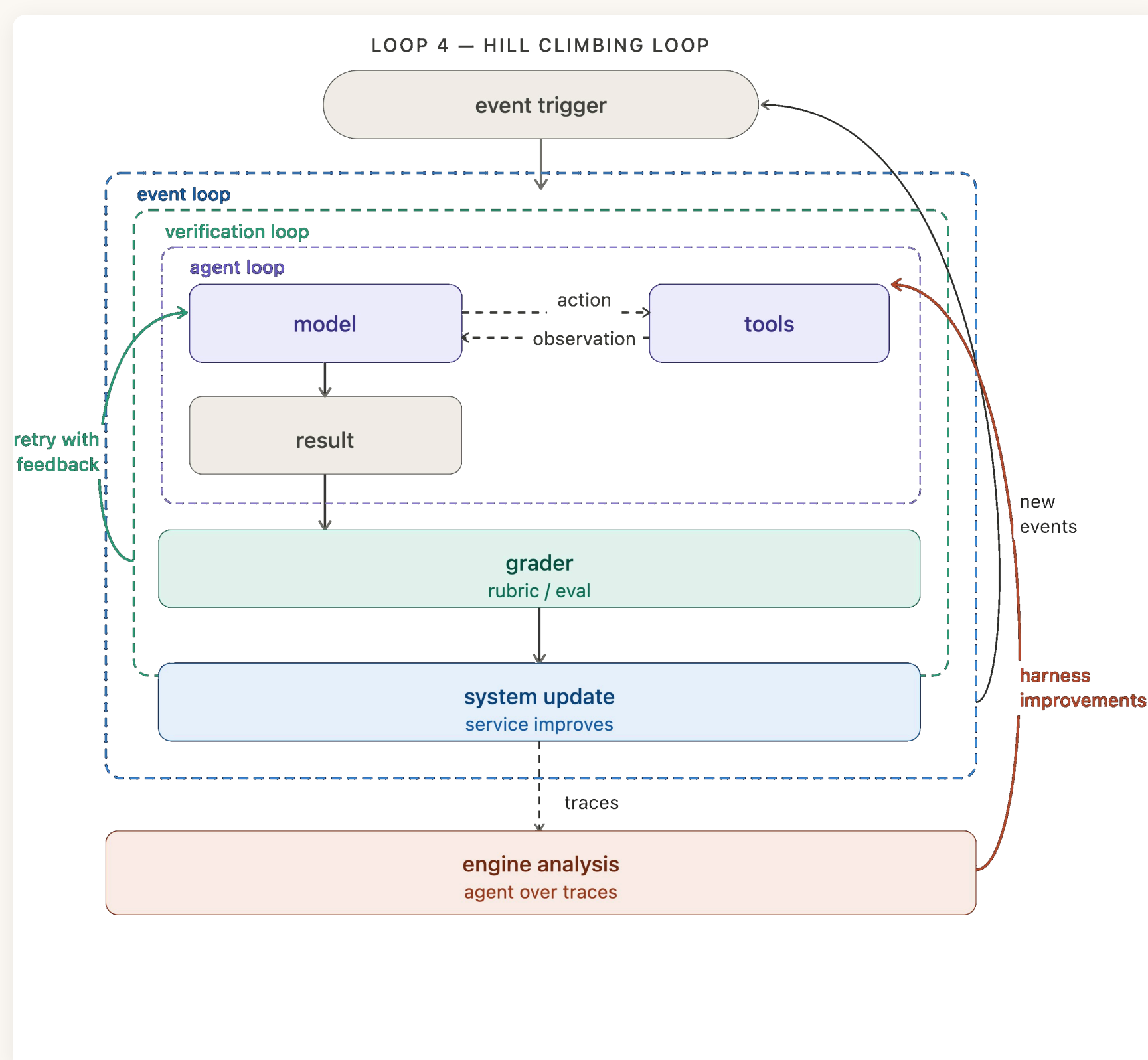
其实本人觉得很多都是 harness engineering 说过的，只是套入到 loop engineering 的场景，所以多了点东西而已。

Q5: loop engineering 也能一层层套娃？

loop也能套娃。LangChain 那篇 《The Art of Loop Engineering》 给的四层栈，从里到外：

1. **Agent Loop (干活)**：模型反复调工具，把活干完。最里层，就是 ReAct 那一圈。
2. **Verification Loop (验活)**：外面套一个 grader，按 rubric 给产出打分，不达标就带着意见打回去重做。
3. **Event Loop (触发)**：再外面是事件触发——不是你手动喊一句，而是系统里发生了什么事（比如 Slack 来了条消息），自动把 agent 拉起来干活，干完的结果又可能触发新一轮新事件。
4. **Hill Climbing Loop (爬坡)**：再跑一个分析 agent，去看前面这些循环留下的记录，然后回头改循环本身——改 prompt、改工具、改打分标准，让系统自己优化自己，有点 meta harness的感觉。

每一层其实都有loop的味道。



LangChain 原图里的四层套娃：最里面是 model↔tools 反复调用的 agent loop，外面套一层 grader 打分的 verification loop，再外面是事件触发、能把结果喂回系统的 event loop，最外层的 engine analysis 拿着全部 trace 回头改前三层——对应上面 1-4 层（图源：LangChain 《The Art of Loop Engineering》）

LangChain 用了一个**文档维护 agent**作为例子说明：Agent Loop 收到"改一下这篇文档"的请求，自己 clone 仓库、读文件、写文档、开 PR；Verification Loop 每次改完自动跑一遍 grader，检查链接能不能打开、CI 是不是绿的、diff 有没有超出这次该改的范围；Event Loop 不用人手动触发，团队 Slack 里 **#docs-plz** 频道一有人发消息就自动拉起这个 agent；Hill Climbing Loop 是另一个 agent 定期把这些运行记录扫一遍，发现同一类问题反复出现，就自动开个 issue 要求改 prompt 或改工具，不用等人盯日志发现规律。

我再举一个例子——让 AI 自己调 prompt，套的也是这四层。

准备一个**测试集**（一批输入，配上 golden answer，或者一套能打分的标准），给一句目标"把这个 prompt 在测试集上的准确率调到 90%+"，剩下的交给 AI 自己在循环里改：

- AI 改一版 prompt、调用 LLM 接口跑一遍、报错了自己修——这是最里层，纯粹的 ReAct 式干活循环，管的是"这一步能不能跑通"，还没涉及"改得好不好"，这是**Agent loop**。
- 跑完一版拿去测试集上打分，对着 golden answer 算准确率，没到 90% 就照着扣分最多的样本继续改，一轮轮循环，直到过线，或者撞上系统设的轮数上限才停。这一层管的是"改得好不好"，这是**Verification Loop**。
- 再往外一层，触发方式从"你手动喊一句"变成"系统自己按钩子触发"——比如每天固定扫一遍产品当天的用户反馈，抓出用户实际问的 prompt 和期待的答案，自动拉起一轮调优，不用你天天盯着，这是**Event Loop**。
- 最外层是让整个调优过程本身更稳——定期回看这些调优跑下来反复出现的失败模式，把教训写成 skill 或 AGENTS.md，其实就是把 Q2 说的 Harness Engineering 那套也塞进了循环里，让下一轮 Agent Loop 少踩同样的坑。这一层同样能定时跑，本身也是个 loop。这是**Hill Climbing Loop**。

四层每层都是一个独立的 loop，也都能被再往外面套一层，层层嵌套上去。但套娃不是越多越好，每多一层都是额外的成本和复杂度。

Q6：循环跑起来后，人还要在里面干什么？

每来一种新的 AI 用法，第一个该问的都是同一个问题：**人和 AI 的分工，这回又挪到哪了？**把 Q2 那四层拿过来对一遍，是同一条主线：模型能扛的活越来越多，人退得越来越靠后，但退到哪一步都不是“没事干”，只是干的事变了性质、颗粒度变粗了。

- **Prompt Engineering 时代，每个人都是 prompt engineering。** 模型只够单轮问答，人得把每一次输入的话调到最顺——怎么措辞、给不给例子、拆不拆步骤，都是人一句句手动磨出来的。答案好不好，也是人当场肉眼判，人在这一阶段就是 prompt 工程师。
- **Context Engineering 时代，人开始搭建 agent 组件。** 模型能做简单多轮 agent 任务了，人要去管“模型这一步能看到什么”——RAG 检索什么、MCP 接哪些系统、message history 怎么裁剪才不撑爆窗口。人要去“控制信息流”，要开始搭建各种上下文插件。
- **Harness Engineering 时代，人开始把经验写成自动检查的规则。** 不再指望“模型看到对的信息就会做对”，人干的事变成写 linter、搭 test gate、建 observability——以前是人肉眼盯着揪错，现在是把“这里容易错”这件事写成一条规则。
- **Loop Engineering 时代，人是定目标、提供反馈的。** 从“一行行写代码”，变成“把任务说清楚 → 让循环跑 → 审产出 → 给反馈 → 直到满意”。

可以看到，验证永远是你的事。循环会同时放大好和坏——一步错没人拦，后面就层层叠加，最后给你一个看着挺完整、其实满是 bug 的东西。

Addy Osmani：同一个循环，你拿它去加速一件你本来就懂的活，和拿它去躲开“搞懂”这件事，结果是反的。这其实就是**理解债**：用 loop engineering 产出代码的速度会越来越快，可你对代码的理解没跟上，这笔债早晚要还。这套东西不是让你撒手不管，而是把你对工程的理解写进循环里，让它跑得更快。

参考资料（按发布时间排序）

1. [Loop Engineering](#) (Addy Osmani 个人博客) —— 2026-06-07
2. [What Is Loop Engineering? The New Meta for AI Coding Agents](#) (MindStudio) —— 2026-06-09
3. [Loop Engineering: A Guide for Engineers and Practitioners](#) (Adnan Masood, Medium) —— 2026 年 6 月（具体日期未标注）
4. [The Art of Loop Engineering](#) (LangChain) —— 2026-06-16

5. [What Is Loop Engineering? A Complete Guide from Prompt to Harness Engineering \(2026\) \(Tosea\)](#) —— 2026-06-16
6. [Loop Engineering: How to Build AI Agent Loops That Run Themselves \(Requesty\)](#) —— 2026-06-17
7. [Loop Engineering \(Addy Osmani, O'Reilly Radar 转载版\)](#) —— 2026-06-22
8. [Three Key Loops for Building Great Software \(DeepLearning.AI The Batch\)](#) —— 2026-06-26
9. [Loop Engineering又是啥? 一文讲清企业Agent落地的四层工程进化论 \(李伟山, 腾讯云开发者\)](#) —— 2026-07-02
10. [Getting started with loops \(Anthropic / ClaudeDevs\)](#) —— 2026-07-07